



Let's review the design and recreate it.

Outcome expected :

Design diagrams,

Architecture diagrams,

Flow diagrams,

APIs system design diagrams,

Low level design diagrams with evaluation logic,

Evaluation - detailed calculated and reasoning documentation.

Project requirement for design implementation :

We are here to design project about evaluation module, this evaluation module should help users understand what kind of RAG pipeline and ingestion pipelines can be set with in detailed suggestion of parameters to pick for optimal utilisation and best outcomes from RAG pipeline.

1. What embedding model to use and how to recommend which model to use, static and dynamic evaluation approach for the same. With parameters for embeddings with static and dynamic way of evaluation and logic of doing the evaluation.
2. What chunking strategy to use and optimised parameters about chunking like overlap, size, merge small chunks, normalisation and more properties that can be configured in chunking strategy. Need to provide static way of evaluation logic and dynamic logic for the way of evaluating and finding the right parameters.
3. What dimensions to use without losing much of data and context and the intention is to save space with best results based on RAG implementation usecases. Need static way of evaluation logic and dynamic way of evaluation logic.
4. What LLM model to use for RAG inference and the evaluation logic both static evaluation and dynamic evaluation based on usecase.
5. What indexing to use on top of embeddings created in vector databases like mongodb, qdrant, Melvis, redis and faiss. Logic of how to choose the index and detailed analysis and logic to do by both static and dynamic way of evaluating and finding the index.
6. What vectordb to be used. Both static and dynamic evaluation logic to choose which vectordb to use.
7. What retrieval mechanism to use, based on different vector databases suggested as per point 6, what is the best retrieval mechanism and parameters to choose based on usecase and data. Need to outline static way of suggesting the parameters and dynamic way if evaluating and suggesting the right parameters and values for it.
8. Prompt template to use based on usecase and best possible templates evaluation and suggestion.

Design approach suggestion to think in line,

All 8 static & dynamic evaluations outlines above should have API endpoints. Need to detail the input load and output expectation for all the APIs

Over all recommended API, which uses these 8 APIs internally and recommend complete pipeline for RAG or ingestion.

System must use multiple iterations of calculation in search of best ways for parameters in all 8 evaluation requirements. Those iteration logic should be detailed and called out.

Input:

Need to list out detailed information that is needed from user so that evaluations can be done for all the required parts of ingestion and RAG pipeline sub modules like the 7 provided above.

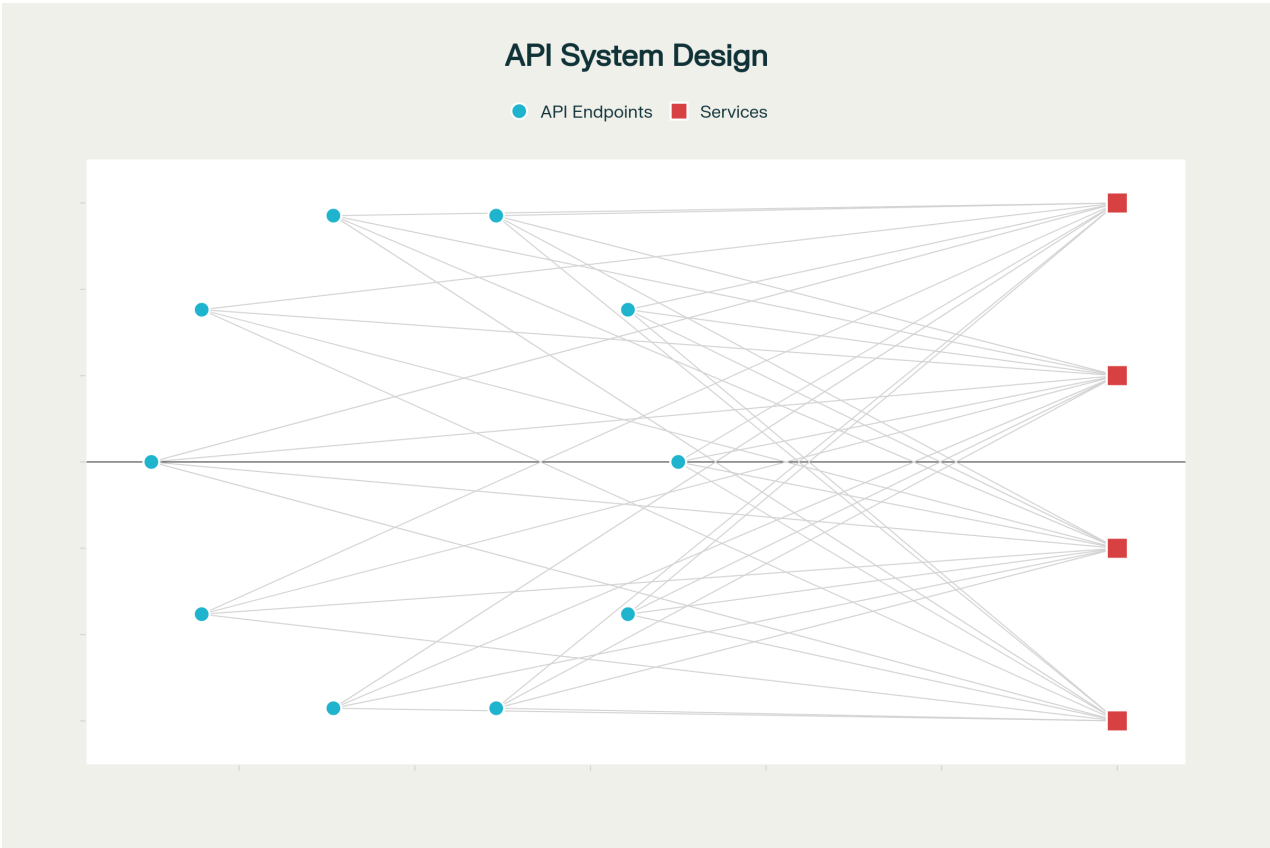
Which might be but not limited to asking

1. Source data type like pdf, word, txt, rdbms tables, nosqls, and so on
2. Sample data file or tables to be used for evaluation.
3. RAG Use case details,
4. Volume of data and increases in volumes in future
5. Cost for RAG usage (optional)
6. Priorities like Accuracy followed by latency followed by cost or any other way.
7. Data privacy
8. Complexity of queries
9. Latency time allowed
10. Accuracy tolerance
11. Truthfulness tolerance

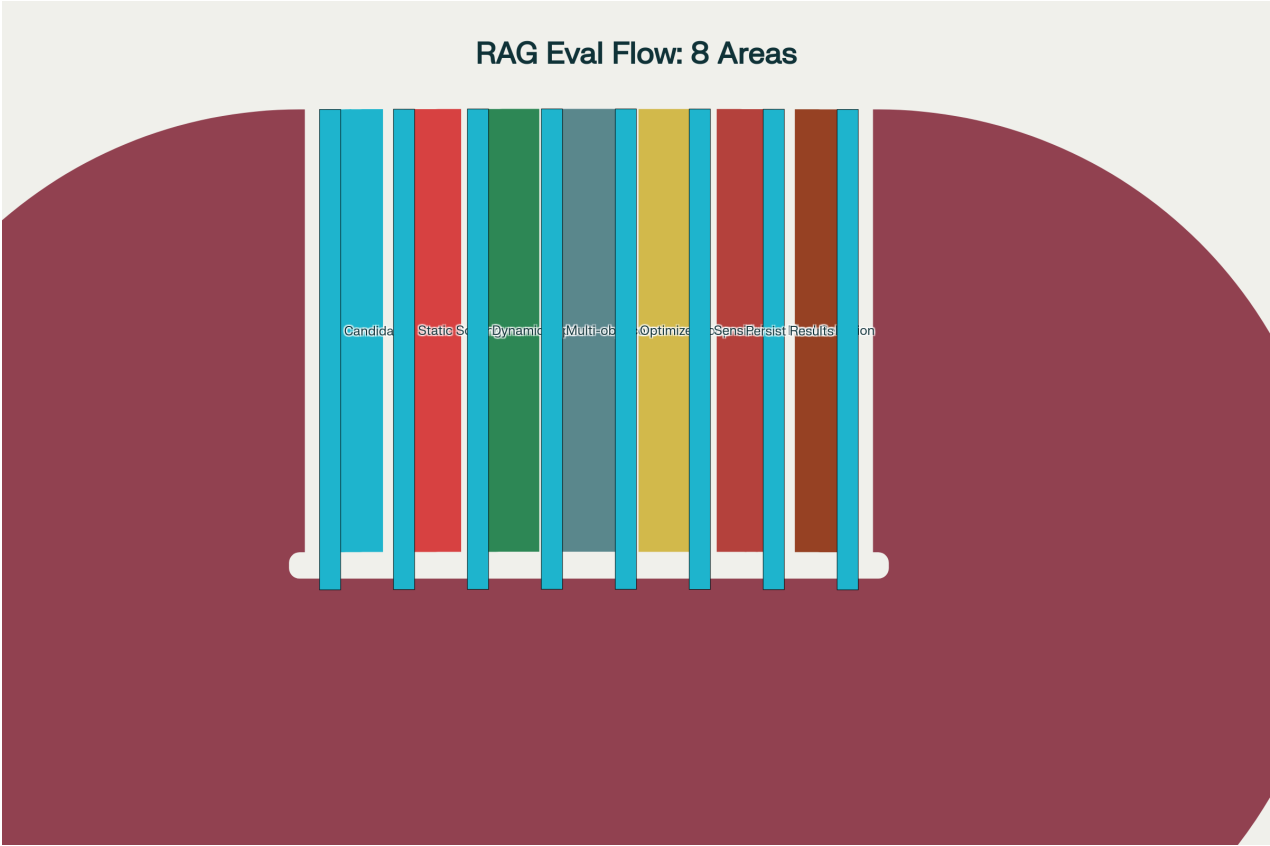
And so on, which would influence the RAG pipeline parameters. To be detailed and asked as input.

Use ROGUE, BLEU and other calculations if relevant and for right evaluations of parameters.

Here is a recreated design with complete diagrams, API system design, low-level evaluation logic, and detailed reasoning for static and dynamic recommendations across all eight areas of a RAG system evaluation module, culminating in a single orchestrated pipeline recommendation API.



APIs System Design and LLD



Dynamic Evaluation Loop (All 8 Areas)

Visual overview

The system includes an API-centered architecture with shared services (Orchestrator, Experiment Runner, Metrics Service, Knowledge Base) and nine evaluation endpoints plus a pipeline synthesizer endpoint.

Evaluation uses a multi-iteration loop applied uniformly across embeddings, chunking, dimensions, LLM, indexing, vectordb, retrieval, and prompt, combining static heuristics and dynamic experiments with multi-objective optimization and sensitivity analysis.

Scope and goals

The evaluation module recommends the complete ingestion and retrieval pipeline with parameter-level suggestions optimized for accuracy, latency, cost, and privacy constraints using static priors and dynamic experiments.^[1]

It covers model choices (embeddings and LLM), chunking, dimensionality, vector database and index configuration, retrieval mechanisms, and prompt templates with rigorous metrics including ROUGE, BLEU, faithfulness, and RAG-grounding metrics.^{[2] [1]}

Inputs required

To ensure robust evaluation, the module requests the following inputs and constraints:

- Use case and task family (doc QA, multi-hop QA, customer support, code QA, KB search, multilingual, multi-modal roadmap) to seed candidate generation and priors.^[1]
- Data sources: formats (pdf, docx, txt, html, web, rdbms, nosql), schemas, metadata fields available for filtering, and access methods for sampling.^[1]
- Sample data: 100–1,000 representative items with ground-truth QA where available for supervised metrics; otherwise reference-free evaluation is used.^{[2] [1]}
- Volume today and forecast (docs, chunks, vectors, QPS, storage) to inform index and database selection and scaling plan.^{[3] [4]}
- Priorities: weights for accuracy, latency, cost, and privacy (e.g., accuracy 0.5, latency 0.2, cost 0.2, privacy 0.1) to drive multi-objective scoring.^[1]
- Latency SLOs and tail behavior constraints (e.g., p95 < 500 ms) used as hard constraints in optimizer loops.^[1]
- Cost bounds (optional) including inference budget per 1k tokens and storage budget per GB per month for feasible candidate pruning.^[1]
- Privacy constraints and data residency requirements to filter models and managed services as needed.^[1]
- Query complexity distribution (simple factoid vs long-form reasoning, multi-hop, code snippets) to choose chunking, reranking, and LLM sizing.^[1]
- Accuracy tolerance and truthfulness tolerance to set metric thresholds for recall@k, faithfulness, and hallucination rates.^{[2] [1]}

APIs system design

The evaluation module exposes 10 endpoints: 8 domain-specific evaluators, a user requirements intake, and a pipeline synthesizer.

Endpoints, inputs, outputs

Endpoint	Purpose	Key inputs	Outputs	
POST /user-requirements	Capture all inputs and constraints	use_case, priorities, data_sources, samples, constraints	requirements_id	
POST /evaluate/embeddings	Recommend embedding model and params	requirements_id, candidates, eval_mode(static	dynamic)	best_embedding, params, scores
POST /evaluate/chunking	Recommend chunking strategy and params	requirements_id, candidates, eval_mode	best_chunking, params, scores	
POST /evaluate/dimensions	Recommend target dimensionality	requirements_id, target_metrics, eval_mode	best_dimension, method, tradeoffs	
POST /evaluate/llm	Recommend LLM and inference params	requirements_id, candidates, eval_mode	best_llm, params, scores	
POST /evaluate/indexing	Recommend index per vectordb	requirements_id, vectordb, candidates, eval_mode	best_index, params, scores	
POST /evaluate/vectordb	Recommend vector database	requirements_id, candidates, eval_mode	best_vdb, config, scores	
POST /evaluate/retrieval	Recommend retrieval mechanism & params	requirements_id, candidates, eval_mode	best_retrieval, params, scores	
POST /evaluate/prompt	Recommend prompt templates	requirements_id, templates, eval_mode	best_prompt, variables, scores	
POST /recommend/pipeline	Synthesize complete pipeline	requirements_id	integrated_pipeline, configs, report	

Evaluation loop (static + dynamic)

The evaluator follows a common flow for all eight areas: generate candidates → static scoring → dynamic experiments → multi-objective scoring → optimization loop → sensitivity analysis → selection with rationale → persist results.

Static scoring uses priors from benchmarks and docs; dynamic experiments run A/Bs, holdouts, or cross-validation on samples to measure retrieval, generation, and end-to-end metrics; multi-objective scoring merges accuracy, latency, cost, and privacy per priority weights; and the optimizer uses Bayesian/grid/random search with early stopping and down-selection.^{[1] [3]}

A normalized utility score aggregates objectives: $U = w_a \cdot \text{norm}(\text{accuracy}) - w_l \cdot \text{norm}(\text{latency}) - w_c \cdot \text{norm}(\text{cost}) - w_p \cdot \text{penalty}(\text{privacy})$ with weights from priorities and penalties for violations.

1) Embedding model evaluation

- Static evaluation: shortlist by task fit, multilingual need, context length, and licensing; seed with families like OpenAI text-embedding-3-large, MPNet/ MiniLM variants, and domain-specific encoders based on MTEB trends and vendor benchmarks.^{[2] [4]}
- Dynamic evaluation: compute retrieval-quality proxies with fixed retriever and chunking, measuring recall@k, MRR, and NDCG over sampled queries and compute embedding throughput and cost per 1k docs.^{[1] [4]}
- Metrics: retrieval (recall@k, MRR), generation (ROUGE, BLEU), faithfulness/grounding (RAG triad, RAGAS), and latency/cost per candidate.^{[2] [1]}
- BLEU scoring formula: $\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$ with brevity penalty BP and modified n-gram precisions p_n .^[2]

Recommendation logic picks highest utility U while checking constraint satisfaction; e.g., if accuracy priority dominates and latency SLO is met, prefer higher-capacity encoders; if cost is tight, down-select to 384–768D encoders with batching tuned for throughput.^{[1] [2]}

2) Chunking strategy evaluation

- Static evaluation: choose strategy by content structure—recursive paragraph/sentence for mixed text, semantic chunking for narrative, section-aware for structured technical docs, and windowed fixed-size for conversational logs; set initial sizes 512–1500 and overlap 50–200 with “merge small chunks” enabled.^[1]
- Dynamic evaluation: sweep chunk_size, overlap, and normalization toggles; evaluate context relevance and recall@k versus latency and token costs; if “context relevance” is low, adjust parsing and chunk sizes per AWS guidance.^[1]
- Metrics: context relevance, retrieval recall@k, context hit-rate, tokens-per-query, latency; generation: ROUGE/BLEU deltas under chunk variations to quantify impact on answer quality.^{[2] [1]}

The optimizer loop uses random/grid then Bayesian refinement to converge on chunk_size/overlap under SLOs; sensitivity analysis checks robustness versus query types and

document lengths.^[1]

3) Dimensions selection evaluation

- Static evaluation: set target dimensionality by use case; 768D for semantic search balance, 384D for similarity/latency focus, 1024D for classification/fine-grained boundaries with PCA or model-native reduction candidates.^[2]
- Dynamic evaluation: run PCA-variance sweeps (e.g., retain 90–99%) and measure end-to-end retrieval/generation deltas plus index/query speed and storage footprint; select the “knee” point with minimal accuracy loss for maximal storage/latency gains.^{[1] [3]}
- Rationale: modern guidance supports tuning embedding dimensionality to optimize performance-resource ratios instead of always using maximal dimensions.^{[3] [2]}

4) LLM selection for inference

- Static evaluation: shortlist by task complexity, privacy, cost, and latency (e.g., small instruct models for low-latency answers; mid-tier for balanced; larger models or tool-use-augmented models for complex reasoning and high faithfulness needs).^{[2] [1]}
- Dynamic evaluation: run answer groundedness checks (RAG triad: context relevance, grounding/faithfulness, answer relevance) with LLM judges or cross-encoders, plus ROUGE/BLEU and human-in-the-loop spot checks for subtle hallucinations.^{[1] [2]}
- If faithfulness is low, revisit prompt design or increase reranking/constraint mechanisms; if relevance is low, revisit retrieval configs or query rewriting, per the AWS loop.^[1]

5) Indexing strategy evaluation (per vectordb)

- Static evaluation: choose classes—HNSW for in-memory high recall/latency, IVF for large-scale with controllable probe counts, IVF+PQ/OPQ for aggressive memory reduction at some recall penalty; combine selection with db capabilities.^{[5] [6]}
- Dynamic evaluation: sweep HNSW (M, efConstruction, efSearch), IVF (nlist, nprobe), and PQ codebooks; evaluate recall-latency-cost curves using small-to-medium samples; prefer HNSW when RAM is sufficient and IVF/PQ for scale or disk-backing requirements.^{[6] [5]}
- Findings in literature show hybrid sparse+dense fusion and reranking can outperform standalone retrievers, influencing index and candidate set sizes for re-ranking stages.^{[7] [8]}

6) Vector database selection evaluation

- Static evaluation: shortlist by constraints (managed vs self-hosted, filtering/metadata, hybrid search support, cost model), capability fit (HNSW/IVF/PQ, scalar and payload filtering, hybrid sparse+dense), and operational preferences.^{[3] [4]}
- Dynamic evaluation: benchmark chosen candidates (e.g., Milvus, Qdrant, Pinecone, Redis, pgvector) on the actual workload with VectorDBBench-style methodology to avoid misleading synthetic-only results.^{[4] [3]}
- Decision: pick DB with highest utility U for required recall at target latency and budget, then lock index class and parameters discovered in the dynamic benchmarking loop.^{[3] [4]}

7) Retrieval mechanism evaluation

- Static evaluation: consider dense-only, sparse-only (BM25), hybrid fusion (RRF/log-linear), and re-ranking stages (cross-encoder or ColBERT) given data type and query complexity; hybrid often improves recall significantly at comparable latencies when parallelized. ^{[7] [8]}
- Dynamic evaluation: tune top_k, hybrid mixing parameters, sparse analyzer and BM25/BM25L configs, and re-ranker cutoff and batch sizes; measure recall@k, NDCG, latency, and cost; add SPLADE or ColBERT if terms or token-level matching helps domain. ^{[9] [7]}
- RRF scoring formula:
$$\mathrm{RRF}(d) = \sum_{m=1}^M \frac{1}{k + \mathrm{rank}_m(d)}$$
 where k controls the influence of lower ranks across M rankers. ^[8]

8) Prompt template evaluation

- Static evaluation: seed templates (e.g., retrieval-aware system prompt, answer constraints, grounded citation requirements, refusal policies) mapped to use case and tone; define template variables for context windows and safety rules. ^{[2] [1]}
- Dynamic evaluation: A/B templates with the same retrieval outputs; measure answer relevance, faithfulness, consistency (LLM judge), and ROUGE/BLEU vs references; select prompts that improve faithfulness and correctness without inflating tokens or latency. ^{[1] [2]}
- If faithfulness drops, strengthen instructions and apply response validation or self-check prompts; if verbosity exceeds SLOs, compress prompt and enforce concise styles. ^[1]

Dynamic multi-iteration logic

- Candidate generation: derive a candidate set per domain from priors and capability filters (e.g., available models, indexes) and constraints (privacy/cost). ^[1]
- Static scoring: apply rules and priors from benchmarks and docs to rank candidates before experimentation and shrink the dynamic search space. ^{[2] [4]}
- Dynamic experiments: run holdout or rolling evaluations with representative queries and content to collect retrieval, generation, latency, and cost metrics under controlled settings. ^{[3] [1]}
- Multi-objective scoring: compute normalized utility U and enforce SLO constraints; non-dominated fronts are retained for deeper exploration. ^[1]
- Optimizer loop: grid/random for coarse search, then Bayesian or bandit strategies with early stopping once confidence bounds stabilize; log every trial to the Knowledge Base. ^[3]
- Sensitivity analysis: perturb key params (e.g., top_k, efSearch, chunk_size, dimension k) to ensure robustness across realistic variance in inputs and queries. ^[1]

Metrics and calculations

- Retrieval: precision@k, recall@k, MRR, NDCG for candidate configurations and ablations.^[1]^[4]
- Generation: ROUGE-N/L, BLEU, and judge-based scoring for faithfulness and groundedness; combine with hallucination indicators such as consistency-based methods and RAG triad checks.^[2] ^[1]
- System: end-to-end latency (p50/p95), throughput, cost per query, storage footprint, and operational headroom under load.^[3] ^[4]

Orchestrated pipeline recommendation

POST /recommend/pipeline runs all eight sub-evaluations, merges results, and emits a complete configuration and rationale with parameter-level choices tied back to observed metrics and trade-offs.

Outputs include embedding model and parameters, chunking strategy, target dimensions, LLM and inference settings, vectordb and index configuration, retrieval mechanism with reranking, and a production-ready prompt template with variables and guardrails.^[1]

Concrete decision guidance highlights

- Index selection: prefer HNSW when RAM is sufficient for high recall at low latency; prefer IVF/IVF+PQ when scale or memory limits dominate, tuning nlist/nprobe for recall-latency trade-offs.^[5] ^[6]
- Hybrid retrieval: fuse BM25 with dense retrievers via RRF or learned weighting; literature consistently shows improved recall at reasonable overheads, especially with domain-shifted corpora.^[7] ^[8]
- Evaluation loop as feedback: low context relevance → optimize parsing, chunk size, embeddings; low faithfulness → adjust LLM/prompt or add reranking; low answer relevance → prompt/fine-tune or rewrite queries, per AWS guidance.^[1]

What to submit to run evaluations

- Representative corpus samples and optional labeled QA pairs, plus constraints and priorities as described above; the orchestrator will drive static and dynamic loops and return the best configuration with evidence and reasoning.^[1]
- The system persists all trials and metrics for auditability and re-use, enabling continuous improvement as data and requirements evolve.^[3]

References and sources used for the evaluation design, metrics, indexing choices, hybrid retrieval, and benchmarking methodology include Elastic's RAG metric guidance, AWS Bedrock's RAG evaluation loop, Milvus and Redis articles on HNSW/IVF/PQ, Qdrant benchmarks and hybrid search practices, fusion function analyses, and VectorDBBench.^[2] ^[5] ^[6] ^[8] ^[4] ^[1] ^[3]

1. <https://aws.amazon.com/blogs/machine-learning/evaluate-the-reliability-of-retrieval-augmented-generation-applications-using-amazon-bedrock/>
2. <https://www.elastic.co/search-labs/blog/evaluating-rag-metrics>
3. <https://github.com/zilliztech/VectorDBBench>
4. <https://qdrant.tech/benchmarks/>
5. <https://milvus.io/ai-quick-reference/how-does-indexing-work-in-a-vector-db-ivf-hnsw-pq-etc>
6. <https://dev.to/redis/is-your-vector-database-really-fast-i62>
7. <https://arxiv.org/html/2409.01357v1>
8. <https://dl.acm.org/doi/10.1145/3596512>
9. <https://infiniflow.org/blog/best-hybrid-search-solution>
10. <https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation>
11. <https://www.getmaxim.ai/blog/rag-evaluation-metrics/>
12. <https://blog.relari.ai/a-practical-guide-to-rag-evaluation-part-2-generation-c79b1bde0f5d>
13. <https://qdrant.tech/blog/rag-evaluation-guide/>
14. <https://www.truefoundry.com/blog/best-vector-databases>
15. <http://musingsaboutlibrarianship.blogspot.com/2024/06/can-semantic-search-be-more.html>
16. <https://arxiv.org/html/2405.07437v2>
17. <https://arxiv.org/html/2505.07621v1>
18. <https://qdrant.tech/articles/sparse-vectors/>
19. <https://apxml.com/posts/best-databases-for-rag>
20. <https://eyka.com/blog/reranking-in-rag-enhancing-accuracy-with-cross-encoders/>